

CoRIM 101

Part 1

**General principles, info/data model & the CoRIM
processor**

A two-part journey

Part 1 (today)

General principles

Information & data model

MTI triples

The CoRIM processor

Part 2 (next)

OTI triples

Profiles in depth
(BYOT)

What is a CoRIM?

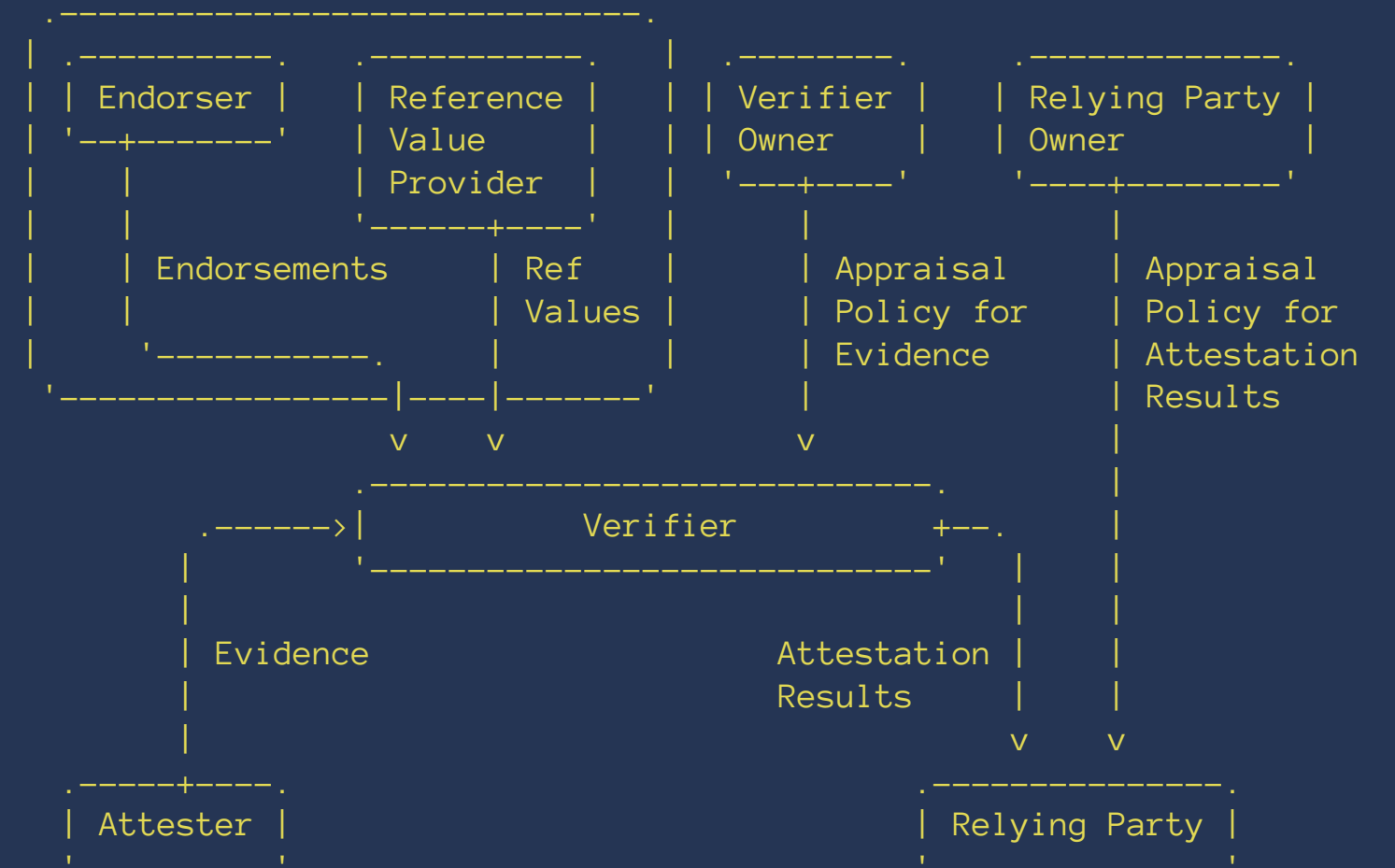
- A **Concise Reference Integrity Manifest** describes the **expected state of an Attester** to a Verifier.
- It is the standard, interoperable container for two RATS inputs:
 - **Reference Values** – what the Attester *should* report
 - **Endorsements** – features *not* visible in Evidence (e.g. certification, provenance)
- Encoded in **CBOR**, signed with **COSE**, extensible via **profiles**.

Goal: a common representation for Endorsement and Reference Value inputs across a diverse supply chain.

Where CoRIM sits wrt RATS roles

Endorser and Reference Value Provider feed the **Verifier**.

CoRIM carries Reference Values + Endorsements.



Reference state vs. actual state

Reference state

What the manufacturer / supply chain says a good Attester looks like.

Carried in CoRIM *Reference Values*

Actual state

What *this* Attester actually reports, in *Evidence* (plus anything that can be inferred from CoRIM *Endorsed Values*)

Appraisal

Reconciling actual state against reference state, then augmenting it with endorsed claims.

Two things in one spec

- **Information / data model** (§4–6): the CBOR/CDDL structures – CoRIM, CoMID, triples.
- **Reference Verifier** (§8): a normative *reference* CoRIM Processor and a 4-phase appraisal algorithm.

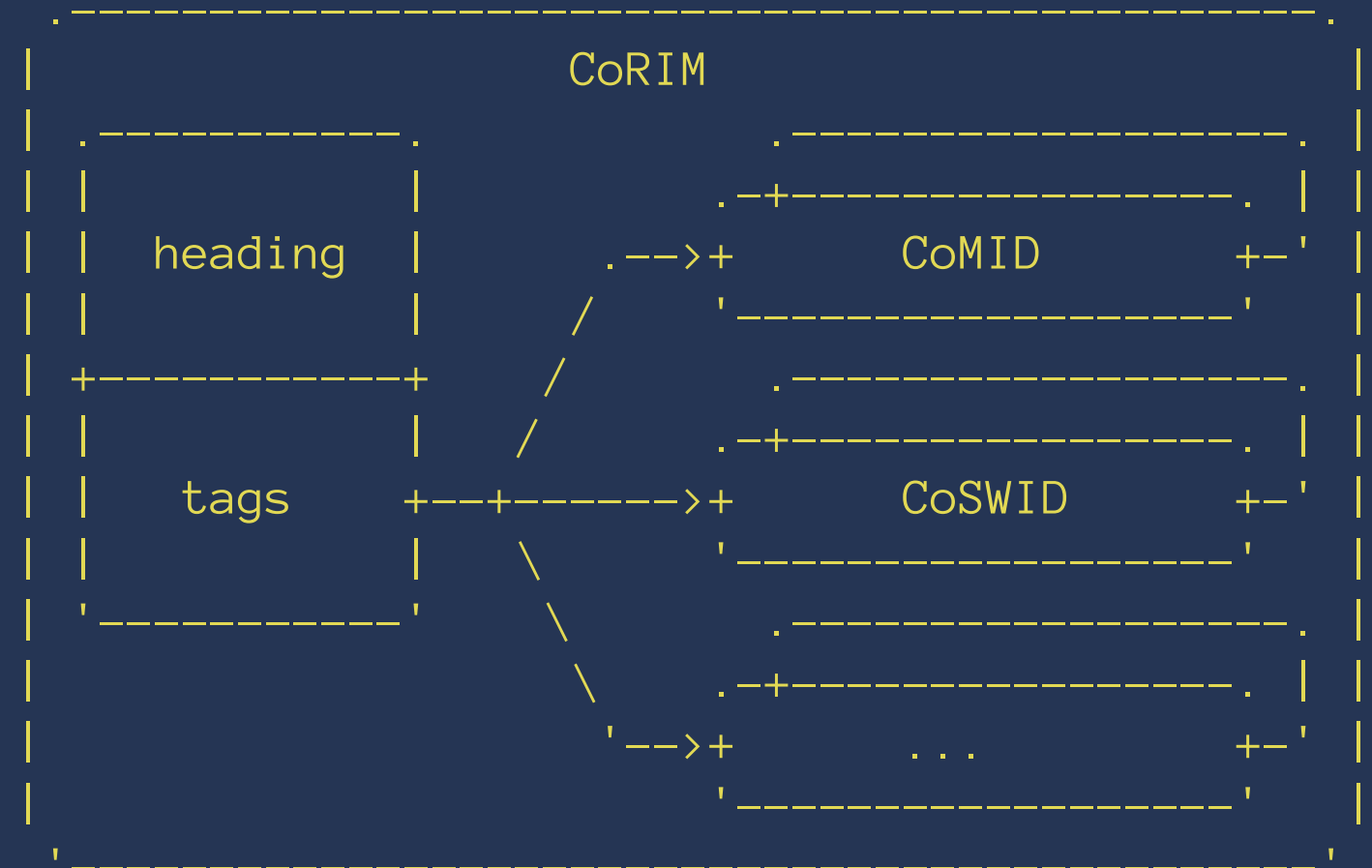
Layer	Technology
Serialization	CBOR (STD94)
Schema	CDDL (RFC8610 & friends)
Authenticity	COSE (STD96)
Media type	application/rim+c{bor,ose}

The data model

CoRIM → CoMID → triples

CoRIM: the outer container

- A **CoRIM** = a heading + a list of **tags**.
- Tags are **CoMID** (#6.506), **CoSWID** (#6.505) or **CoTL** (#6.508).
- The header carries id, profile, validity, entities and dependent-rims.



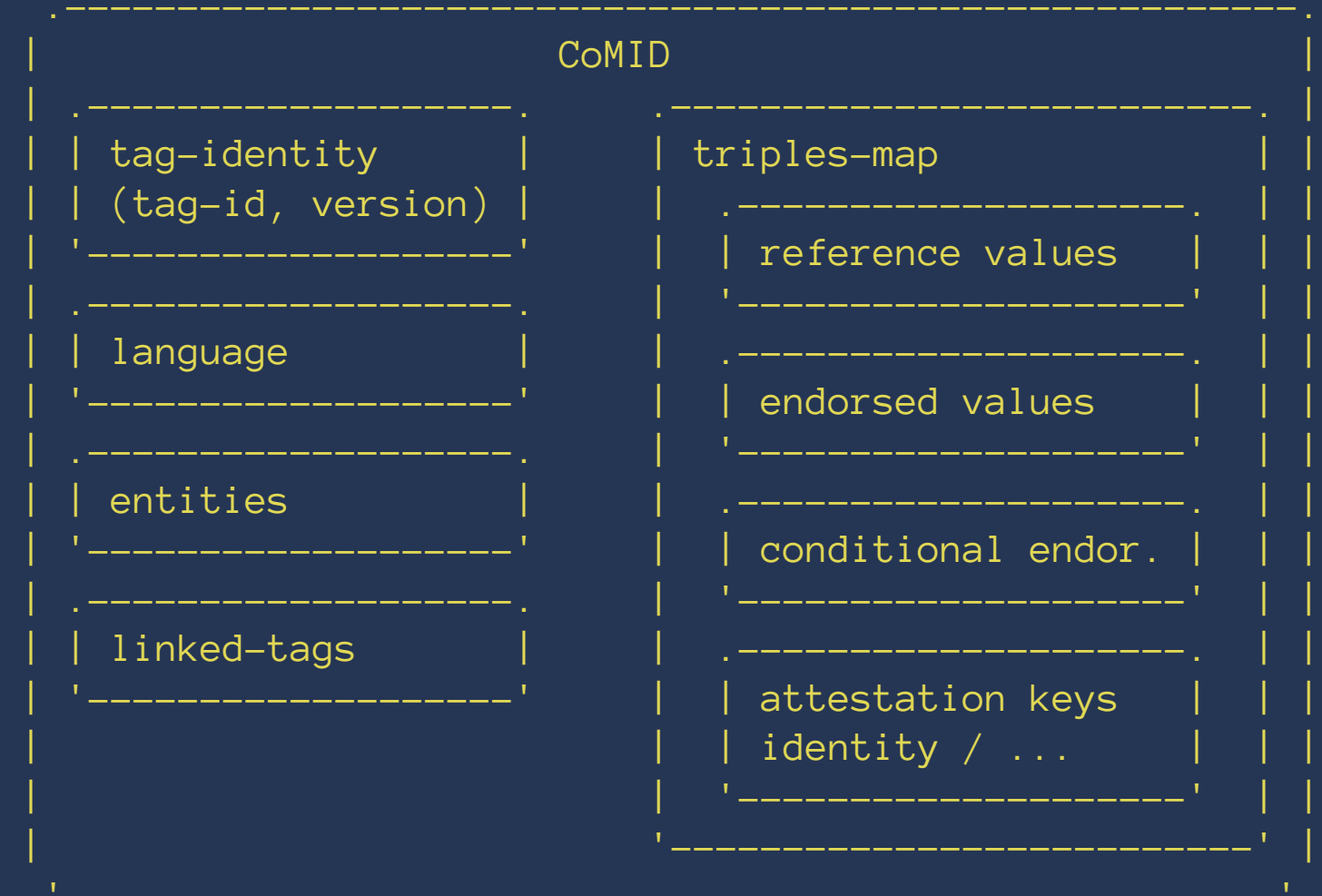
CoRIM map (CDDL)

```
corim-map = {  
    &(id: 0) => $corim-id-type-choice  
    &(tags: 1) => [ + $concise-tag-type-choice ]  
    ? &(dependent-rims: 2) => [ + corim-locator-map ]  
    ? &(profile: 3) => $profile-type-choice  
    ? &(rim-validity: 4) => validity-map  
    ? &(entities: 5) => [ + corim-entity-map ]  
    * $$corim-map-extension  
}
```

A signed CoRIM is a COSE_Sign1 wrapping tagged-unsigned-corim-map (#6.501)

CoMID: the Concise Module ID

- Identifies a module and groups its **triples** by kind.
- The interesting payload lives in the **triples-map** (codepoint 4).



Triples: subject - predicate - object

A **triple** is an assertion about an environment.

```
triples-map = non-empty<{
  ? &(reference-triples: 0)          => [ + reference-triple-record ]
  ? &(endorsed-triples: 1)           => [ + endorsed-triple-record ]
  ? &(identity-triples: 2)           => [ + identity-triple-record ]
  ? &(attest-key-triples: 3)         => [ + attest-key-triple-record ]
  ? &(dependency-triples: 4)         => [ + trust-dependency-triple-record ]
  ? &(membership-triples: 5)         => [ + domain-membership-triple-record ]
  ? &(coswid-triples: 6)             => [ + coswid-triple-record ]
  ? &(conditional-endorsement-series-triples: 8) => [ + conditional-endorsement-series-triple-record ]
  ? &(conditional-endorsement-triples: 10)    => [ + conditional-endorsement-triple-record ]
  * $$triples-map-extension
}>
```

All triples share the shape:

environment^{*} – *has/implies* – **claims*

MTI vs OTI¹

Mandatory To Implement

Every CoRIM Verifier **MUST** implement:

- **Reference Values**
- **Endorsed Values**
- **Conditional Endorsement**

Optional To Implement

Verifier **SHOULD** implement:

- Conditional Endorsement **Series**
- Device **Identity** keys
- **Attestation** keys
- **Trust dependency**
- **Domain membership**
- CoMID↔CoSWID **linking**

¹ Profiles may reclassify OTI→MTI, never MTI→OTI. Extension triples MUST be OTI.

The shared vocabulary: environment-map

```
environment-map = non-empty<{  
  ? &(class: 0)      => class-map  
  ? &(instance: 1) => $instance-id-type-choice  
  ? &(group: 2)      => $group-id-type-choice  

```

```
class-map = non-empty<{  
  ? &(class-id: 0) => $class-id-type-choice ; OID / UUID / bytes  
  ? &(vendor: 1)   => tstr  
  ? &(model: 2)    => tstr  
  ? &(layer: 3)    => uint  
  ? &(index: 4)    => uint  

```

Identifies *which* environment a triple talks about – by class, instance, or group.

The shared vocabulary: measurement-map

```
measurement-map = {  
  ? &(mkey: 0)          => $measured-element-type-choice  
  &(mval: 1)            => measurement-values-map  
  ? &(authorized-by: 2) => [ + $crypto-key-type-choice ]  
}
```

```
measurement-values-map = non-empty<{  
  ? &(version: 0) => version-map  
  ? &(digests: 2) => digests-type  
  ? &(svn: 1) => svn-type-choice  
  ? &(flags: 3)  => flags-map  
  ...  

```

The *claims* – digests, versions, SVNs, flags, keys, registers, ... – about a measured element.

Mandatory-To-Implement triples

Reference Values

Endorsed Values

Conditional Endorsement

Reference Values

```
reference-triple-record = [  
  ref-env:      environment-map  
  ref-claims: [ + measurement-map ]  
]
```

- Describes an **acceptable state** of a Target Environment.
- Compared against Evidence during appraisal (Phase 3).
- Ranges/multiple entries express the set of *possible* good states.

Reference Values – example (environment)

Target environment df74ac78–...–8ff9152bbbee of *Gizmo* made by *Acme*:

```
[
  {
    / class / 0: {
      / (uu)id / 0: 37(h'df74ac7859c34686b7e18ff9152bbbee'),
      / vendor / 1: "Acme",
      / model / 2: "Gizmo"
    }
  },
  / [ + measurement-map ] /
]
```

Reference Values – example (claim)

... has da...e1 as a known-good SHA-256 digest:

```
/ ref-claims / [ {  
  / mval / 1: {  
    / digests / 2: [  
      "sha-256",  
      h'da1c4cf99f3c7371329d40a5bbc832822c67f7be5865e0d64a1d8498caca8fe1 '  
    ]  
  }  
} ]
```

Reference Values – example (serialisation)

bytes .cbor reference-triple-record:

```
82a100a300d82550df74ac7859c34686b7e18ff9152bbbee016441636d65
026547697a6d6f81a101a10282677368612d3235365820da1c4cf99f3c73
71329d40a5bbc832822c67f7be5865e0d64a1d8498caca8fe1
```

Reference Values – how it's processed

Each reference-triple-record becomes an **rv-item** (condition + addition):

```
FUNC transform(T: reference-triple-record, signer, profile) -> rv-item {  
    item := rv-item::NEW()  
    item.addition.cmttype = reference-values  
    item.addition.environment = T.ref-env  
    item.condition.environment = T.ref-env  
    item.condition.element-list = mms_to_ems(T.ref-claims)  
    item.addition.authority = signer  
    IF profile: item.addition.profile = profile  
    RETURN item  
}
```

Claims are copied into the ACS only once the condition matches an Evidence ECT.

Endorsed Values

```
endorsed-triple-record = [  
  condition:    environment-map  
  endorsement: [ + measurement-map ]  
]
```

- Adds facts the Attester cannot self-report – e.g., certification, provenance.
- Unconditional-ish: keyed only on the **environment** (identity/class/group).
- Added to the ACS with the **Endorser's** authority (Phase 4).

Endorsed Values – example

Environment Gizmo/Acme $\Rightarrow$ it is version 1.0.0, svn 1:

```
[
  { / class / 0: {
    / vendor / 1: "Acme", / model / 2: "Gizmo" } },
  / endorsement / [ {
    / mval / 1: {
      / version / 0: { 0: "1.0.0" },
      / svn      / 1: 552(1)
    }
  } ]
]
```

Conditional Endorsement

```
conditional-endorsement-triple-record = [  
  conditions:  [ + stateful-environment-record ]  
  endorsements: [ + endorsed-triple-record ]  
]
```

```
stateful-environment-record = [  
  environment: environment-map  
  claims-list: [ + measurement-map ]  
]
```

- Endorsements gated on **stateful conditions** – matched against the *actual* state before they apply.
- If **all** conditions are corroborated in the ACS, the endorsements are added.

Conditional Endorsement – example

If Gizmo/Acme has measurement digest da..e1 **then** it has version 1.0.0, svn 1:

```
[
  / conditions / [ [
    { / class / 0: { 1: "Acme", 2: "Gizmo" } },
    [ { / mval / 1: { / digests / 2: [ "sha-256", h'da..e1' ] } } ]
  ] ],
  / endorsements / [ [
    { / class / 0: { 1: "Acme", 2: "Gizmo" } },
    [ { / mval / 1: { / version / 0: {0:"1.0.0"}, / svn / 1: 552(1) } } ]
  ] ]
]
```

The OTI triples (Part 2 preview)

Triple	Purpose
Conditional Endorsement Series	Chained conditions, special matching algorithm
Identity keys	Device credentials (e.g. IDevID)
Attestation keys	Keys that verify Evidence integrity
Trust dependency	Trust relationships between domains
Domain membership	Topology of (sub-)modules
CoSWID linking	Associate a Target Environment with CoSWID tags

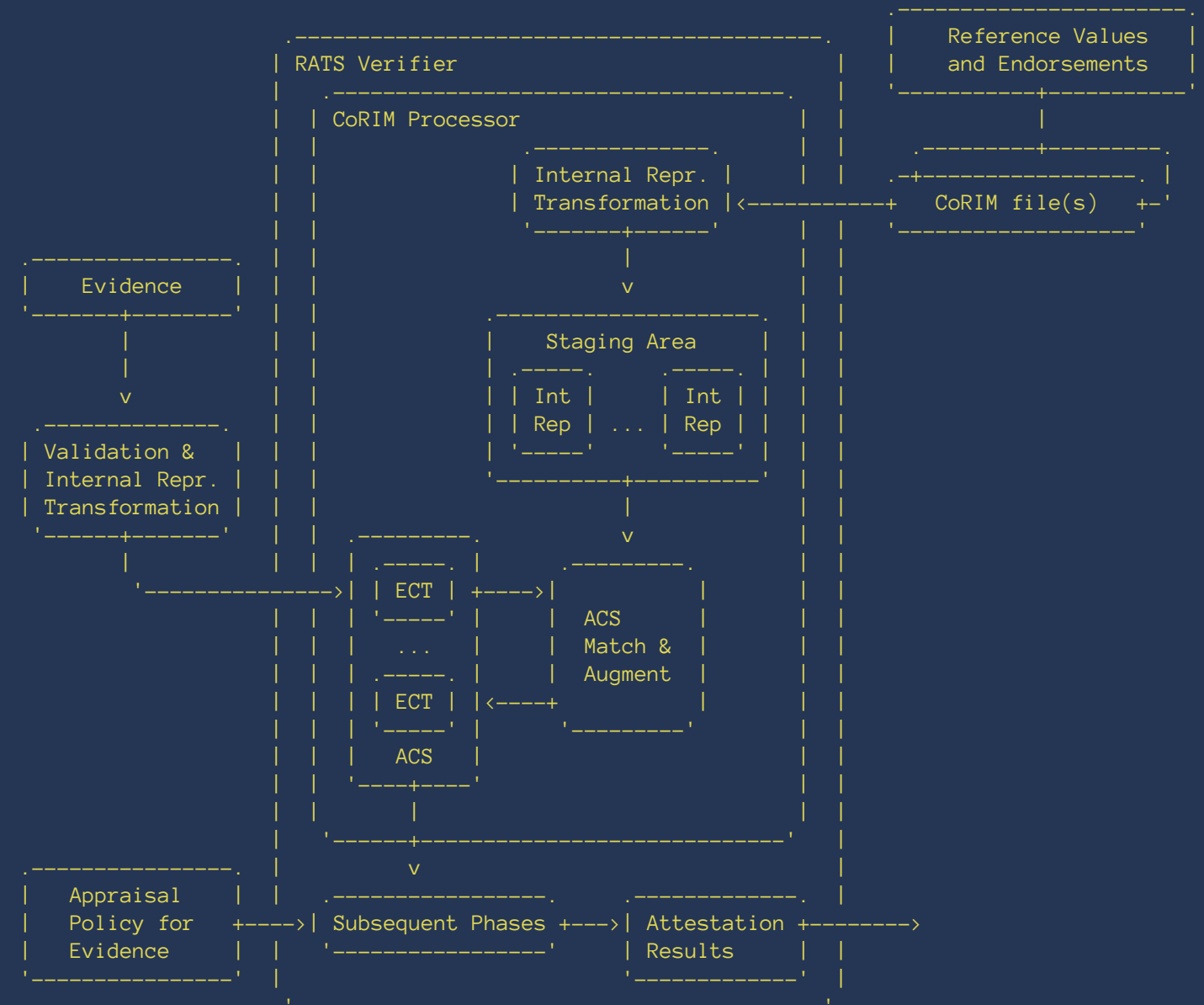
The CoRIM Processor

How reference state meets actual state

Processing flow

Evidence **initializes** the ACS

Reference Values &
Endorsements wait in the
staging area and are
matched in one by one



ECT – Environment-Claims Tuple

The core internal data structure – a statement the Verifier holds about an environment.

```
ECT-common = (  
  ? environment: environment-map  
  ? authority:    [ + $crypto-key-type-choice ]    ; who said it (mandatory in an ECT)  
  ? profile:      $profile-type-choice  
)
```

```
E-ECT = { ECT-common, ? element-list: [ + element-map ], ? cmtype: cm-type }
```

```
cm-type = &( reference-values: 0, endorsements: 1, evidence: 2 )
```

Specializations: **E**-ECT (element), **M**-ECT (membership), **T**-ECT (trust), **K**-ECT (key)

ACS – Appraisal Claims-Set

ACS = [+ ECT]

- The evolving picture of the Attester's **actual state**.
- **Initialized** from Evidence, then **augmented** as staged items match.
- Each added claim is tagged with an **authority** (Attester / RVP / Endorser / Verifier).
- Order of ECTs is not significant.

Relations: condition + addition

The staging area holds *relations* – lists of **items** that pair a matching **condition** with an **addition**.

```
rel      = [ + item ]  
item     = { condition: ECT, addition: ECT }
```

```
ae-item = {          addition }    ; Evidence – no condition (unconditional)  
rv-item = { condition, addition }  ; Reference Values  
ev-item = { condition, addition }  ; Endorsed / Conditional Endorsement
```

- **condition** ECT – the matching criteria, compared against the current ACS
- **addition** ECT – appended to the ACS when the condition matches

Match & augment

Pull each relation from the staging area; match its condition against the ACS; on success, augment.

```
FUNC match_and_augment(acs, sa) -> ACS {  
    FOREACH rel IN sa:  
        FOREACH item IN rel:  
            IF acs::MATCH(item.condition): ; vs. current ACS state  
                acs::APPEND(item.addition) ; augment (atomic)  
        RETURN acs  
}
```

This is where the `rv-item` built by `transform()` lands: on a match the Evidence ECT's `element-list` is copied into the (initially empty) `addition` and re-asserted under **RVP** authority.

What does MATCH mean? Sub-shape matching²

`acs::MATCH(condition)` asks: **is the condition a *sub-shape* of some ACS entry?**

Formalised as **Attribute Path Containment** – a condition *C* is *contained in* an entry *A* when:

1. every **Attribute Path** defined in *C* is also defined in *A*, and
2. at each such path, *A*'s value **satisfies** *C*'s value.

*Attributes present in **A** but not in **C** are ignored – the condition only pins down what it cares about.*

*An **Attribute Path** selects a nested value by successive attribute names, e.g. `environment → class → class-id`.*

²Terminology from the current editor's copy (*Attribute Path Containment*).

Sub-shape matching in action

Condition C (the reference "shape"):

```
{ environment:
  { class: { vendor: "Acme",
               model:  "Gizmo" } },
  element-claims:
    { svn: min-svn(3) } }
```

ACS entry A (Evidence):

```
{ environment:
  { class: { vendor: "Acme",
               model:  "Gizmo",
               layer:  0 },
    instance: h'07..18' },
  element-claims:
    { svn: 5, digests: [...],
      version: {...} } }
```

✓ every path in C is defined in A · svn
5 satisfies min-svn **3** · A's extra
attributes ignored ⇒ **match**

"Satisfies": more than equality

Same containment shape, but the leaf comparison is **type-specific**:

Attribute	A satisfies C when ...
default	values are equal
svn + tagged-min-svn	$A.svn \geq C$ minimum bound
int-range	A value within [min, max]
digests	values equal for every common algorithm
integrity-registers	C may cover a subset of registers (e.g. some PCRs)

The same *sub-shape containment* rule drives environment, element-list, element-claims and key matching alike.

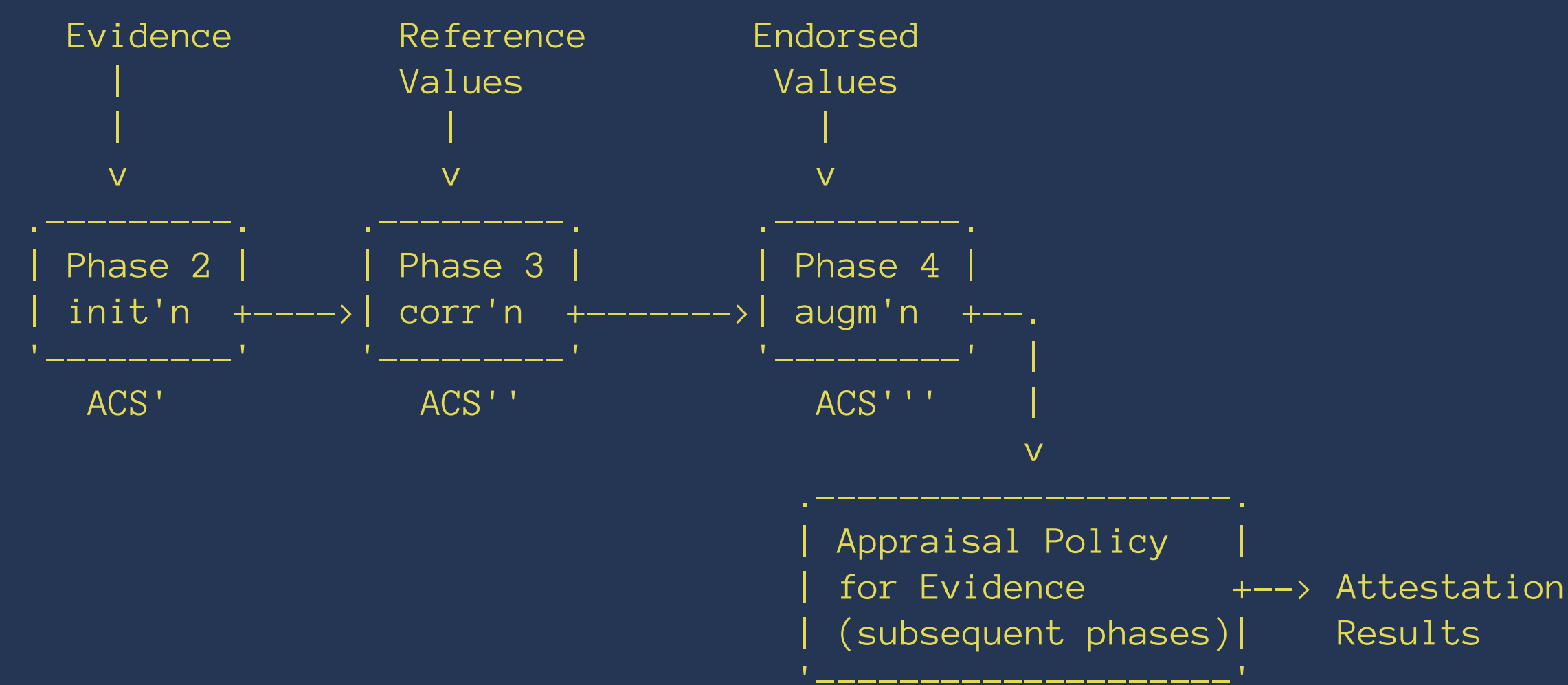
The four appraisal phases

1. **Input Validation & Transformation** – verify signatures/validity; reshape every Conceptual Message into ECTs in a staging area.
2. **Evidence Augmentation** – initialize the ACS from Evidence (Attester authority).
3. **Reference Values Corroboration & Augmentation** – match reference conditions; corroborated claims added with **RVP** authority.
4. **Endorsed Values Augmentation** – apply (conditional) endorsements; claims added with **Endorser** authority.

Subsequent phases: Appraisal Policy for Evidence → Attestation Results.

Life of an ACS

Each phase augments the actual state, then the policy turns it into Attestation Results



Authentic provenance

CoRIMs (and their claims' authorities) are cryptographically bound:

- **COSE_Sign1** over the CoRIM – `application/rim+cose`
- Or embedded via an **X.509 extension** (`tcg-dice-endorsement-manifest`).
- Every ECT carries the **authority** key material establishing who asserted it.

Flexing the data model: profiles

The CoRIM data model is deliberately **generous** – each attestation technology uses it its own way.

Profiling is the built-in mechanism to:

- **Extend** – define (and register) new types / codepoints.
- **Restrict** – declare which semantic subset is in use.

An in-band **profile** (URI or OID) in the CoRIM header tells the receiver how to interpret it.

Summary

- CoRIM = interoperable container for **Reference Values + Endorsements** (CBOR/COSE).
- The data model is **triples** grouped in **CoMID**, over a shared environment / measurement vocabulary.
- **MTI**: Reference Values, Endorsed Values, Conditional Endorsement.
- The **CoRIM Processor** reconciles actual vs reference state across **4 phases**, building an **ACS** of authority-tagged ECTs.
- **Profiles** make it fit any attestation technology.

Next time: OTI triples, profiles & real deployments.

Links & resources

- IETF Datatracker: [draft-ietf-rats-corim](#)
- Editor's copy & issue tracker: [RATS WG on GitHub](#)
- RATS architecture: [**RFC 9334**](#) and [**draft-ietf-rats-endorsements**](#)
- Implementations: **Veraison** CoRIM tooling ([Go package](#), [Rust crate](#), [CLI](#)) & [cover](#) processor; **MSFT Azure** [Rust crate](#).

Questions?